

Ponteiros - Parte 2

Algoritmos e Programação 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Ponteiros: Tipos

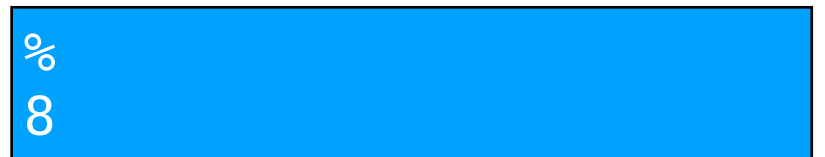
- Qual o tamanho de um ponteiro?

```
#include <stdio.h>

int main () {
    double *p;

    printf("%zu\n", sizeof(p));

    return 0;
}
```



Ponteiros: Tipos (Cont.)

- Qual o tamanho de um ponteiro?

```
#include <stdio.h>

int main () {
    double *p;
    char *test;

    printf("%zu\n", sizeof(p));
    printf("%zu\n", sizeof(test));

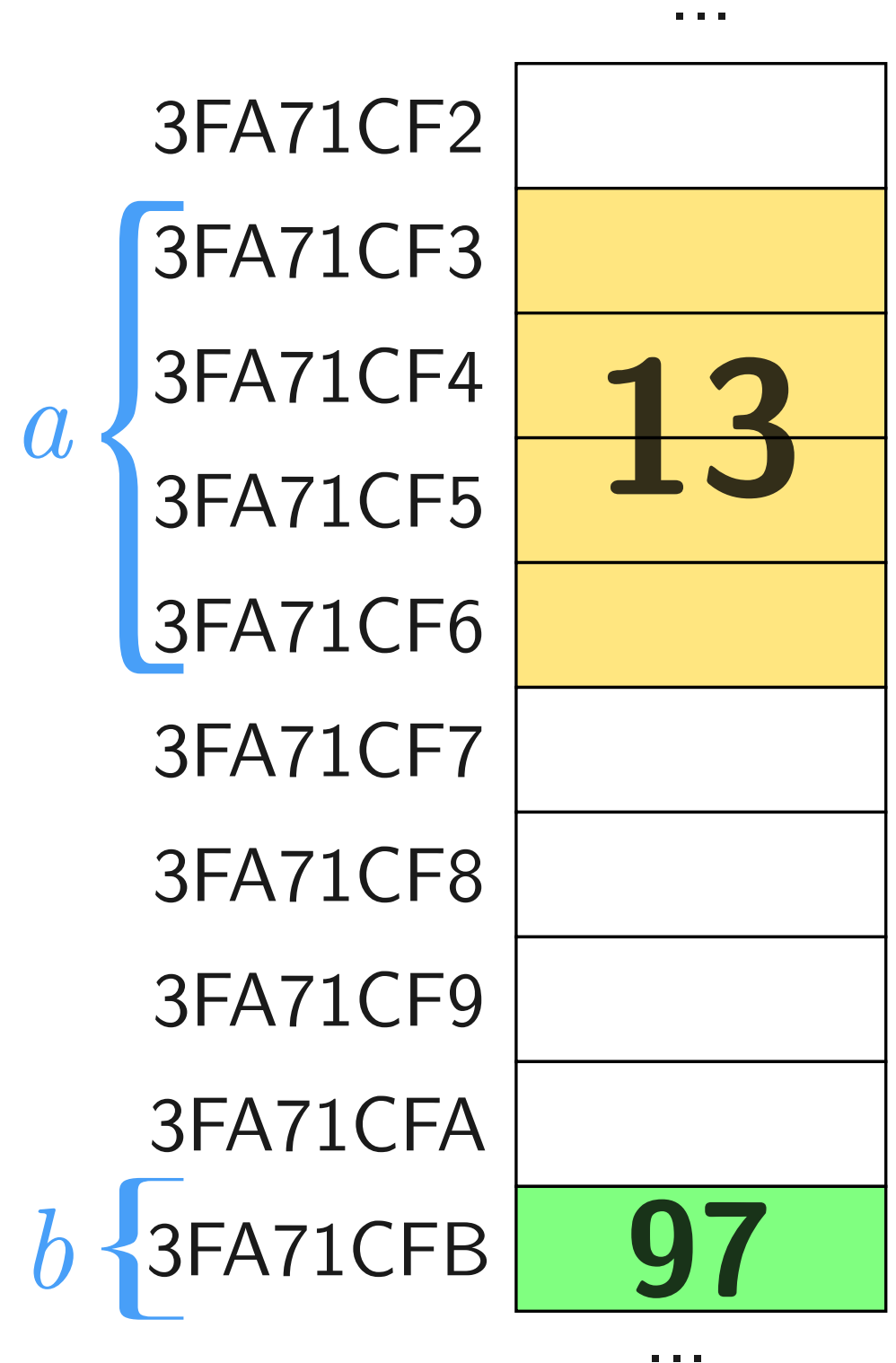
    return 0;
}
```



Tamanho do Ponteiro

```
1. #include <stdio.h>
2.
3. int main () {
4.     int a, *ptr;
5.     char b;
6.
7.     a = 13;
8.     b = 'a';
9.     ptr = &a;
10.    printf("%d\n", a);
11.    printf("%p\n", &a);
12.    printf("%p\n", ptr);
13.
14.    return 0;
15.}
```

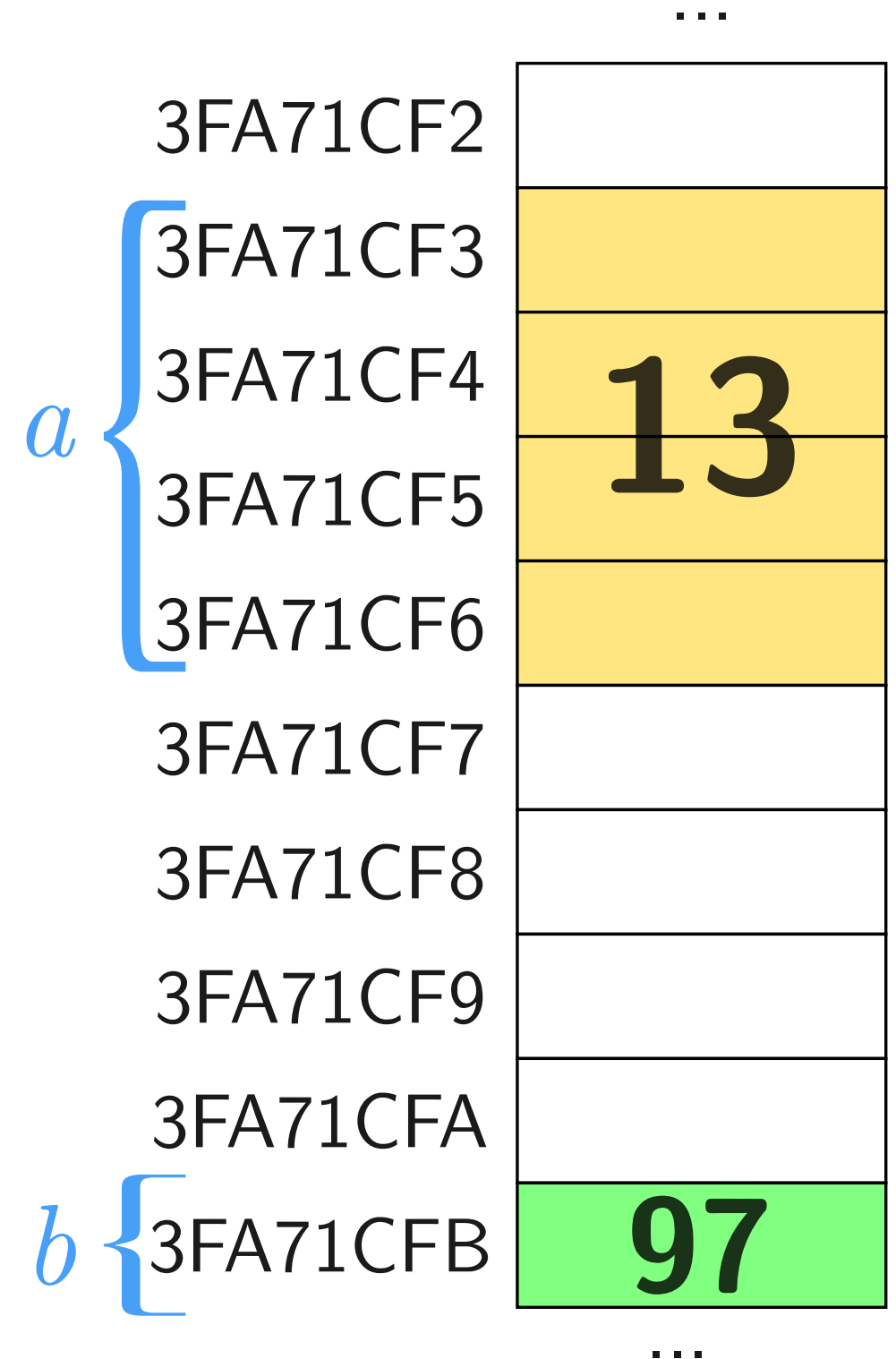
```
%
13
3FA71CF3
3FA71CF3
```



Tamanho do Ponteiro (Cont.)

```
1. #include <stdio.h>
2.
3. int main () {
4.     int a;
5.     char b, *ptr;
6.
7.     a = 13;
8.     b = 'a';
9.     ptr = &b;
10.    printf("%c\n", b);
11.    printf("%p\n", &b);
12.    printf("%p\n", ptr);
13.
14.    return 0;
15.}
```

```
%
'a'
3FA71CFB
3FA71CFB
```



Tipo do Ponteiro

- Por que os ponteiros do tipo **int** e **char** possuem o mesmo tamanho?
 - Ambos armazenam a mesma coisa: um endereço
- Então para que serve o tipo?
 - Serve para quando você referenciar (*), o ponteiro saber **quantos bytes deve considerar** a partir do endereço inicial e como deve ser **interpretado** esses *bytes*

Ponteiro com Tipo Incorreto

```
1. #include <stdio.h>
2.
3. int main() {
4.     int a = 8, *b;
5.     float *c;
6.
7.     b = &a; // b aponta para a variável a
8.     c = (float *)b; // c agora aponta para o mesmo endereço que b,
9.                     // mas como float, o que pode causar problemas
10.
11.     printf("Valor de a: %d\n", a);
12.     printf("Valor apontado por b: %d\n", *b);
13.     printf("Valor apontado por c: %f\n", *c);
14.     // Isso pode causar comportamento indefinido
15.
16.     return 0;
17. }
```

```
%
Valor de a: 8
Valor apontado por b: 8
Valor apontado por c: 0.000000
```


Qualificador Const

Qualificador Const

- Variáveis declaradas com qualificador **const** não podem ser alteradas
 - Constantes!!!
- Declare parâmetros **const** quando desejar que a função não altere o valor (por exemplo de um vetor)
- Tentar alterar uma variável **const** produz um erro

Qualificador Const c/ Ponteiros

```
int *const myPtr = &x;
```

- Ponteiro constante para um **int** (uma vez atribuído um endereço para o ponteiro, não pode ser alterado)

```
const int *myPtr = &x;
```

- Ponteiro para um **const int** (pode-se alterar para onde o ponteiro está apontando, porém não pode-se alterar o conteúdo)

```
const int *const myPtr = &x;
```

- Ponteiro constante para um **const int**

Exemplo 1: Minúsculo p/ Maiúsculo

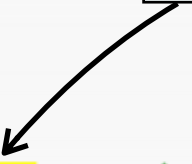
Outline

fig07_10.c

(1 of 2)

```
1  /* Fig. 7.10: fig07_10.c
2     Converting lowercase letters to uppercase letters
3     using a non-constant pointer to non-constant data */
4
5  #include <stdio.h>
6  #include <ctype.h>
7
8  void convertToUppercase( char *sPtr ); /* prototype */
9
10 int main( void )
11 {
12     char string[] = "characters and $32.98"; /* initialize char array */
13
14     printf( "The string before conversion is: %s", string );
15     convertToUppercase( string );
16     printf( "\nThe string after conversion is: %s\n", string );
17
18     return 0; /* indicates successful termination */
19
20 } /* end main */
21
```

Ambos **sPtr** e ***sPtr** são modificáveis



Exemplo 1: Minúsculo p/ Maiúsculo (Cont.)

Outline

fig07_10.c

(2 of 2)

```
22 /* convert string to uppercase letters */
23 void convertToUppercase( char *sPtr )
24 {
25     while ( *sPtr != '\0' ) { /* current character is not '\0' */
26
27         if ( islower( *sPtr ) ) { /* if character is lowercase, */
28             *sPtr = toupper( *sPtr ); /* convert to uppercase */
29         } /* end if */
30
31         ++sPtr; /* move sPtr to the next character */
32     } /* end while */
33
34 } /* end function convertToUppercase */
```

Ambos **sPtr** e ***sPtr** são modificados pela função **convertToUppercase**

The string before conversion is: characters and \$32.98
The string after conversion is: CHARACTERS AND \$32.98

Exemplo 2: Caractere Por Caractere

Outline

fig07_11.c

(1 of 2)

```
1  /* Fig. 7.11: fig07_11.c
2     Printing a string one character at a time using
3     a non-constant pointer to constant data */
4
5  #include <stdio.h>
6
7  void printCharacters( const char *sPtr );
8
9  int main( void )
10 {
11     /* initialize char array */
12     char string[] = "print characters of a string";
13
14     printf( "The string is:\n" );
15     printCharacters( string );
16     printf( "\n" );
17
18     return 0; /* indicates successful termination */
19
20 } /* end main */
21
```

Variável ponteiro **sPtr** é modificável, mas o dado que aponta, ***sPtr**, não é

Exemplo 2: Caractere Por Caractere (Cont.)

```
22 /* sPtr cannot modify the character to which it points,  
23    i.e., sPtr is a "read-only" pointer */  
24 void printCharacters( const char *sPtr )  
25 {  
26     /* loop through entire string */  
27     for ( ; *sPtr != '\0'; sPtr++ ) { /* no initialization */  
28         printf( "%c", *sPtr );  
29     } /* end for */  
30  
31 } /* end function printCharacters */
```

Outline

fig07_11.c

(2 of 2)

sPtr é modificado pela função **printCharacters**

The string is:
print characters of a string

Exemplo 3: Altere const int * const

Outline

fig07_14.c

```
1  /* Fig. 7.14: fig07_14.c
2     Attempting to modify a constant pointer to constant data. */
3  #include <stdio.h>
4
5  int main( void )
6  {
7     int x = 5; /* initialize x */
8     int y;     /* define y */
9
10    /* ptr is a constant pointer to a constant integer. ptr always
11       points to the same location; the integer at that location
12       cannot be modified */
13    const int *const ptr = &x;
14
15    printf( "%d\n", *ptr );
16
17    *ptr = 7; /* error: *ptr is const; cannot assign new value */
18    ptr = &y; /* error: ptr is const; cannot assign new address */
19
20    return 0; /* indicates successful termination */
21
22 } /* end main */
```

Nem o ponteiro **sPtr** nem o dado que aponta, ***sPtr**, é modificável

```
Compiling...
FIG07_14.c
c:\books\2006\chtp5\Examples\ch07\FIG07_14.c(17) : error C2166: l-value
specifies const object
c:\books\2006\chtp5\Examples\ch07\FIG07_14.c(18) : error C2166: l-value
specifies const object
Error executing cl.exe.

FIG07_12.exe - 2 error(s), 0 warning(s)
```

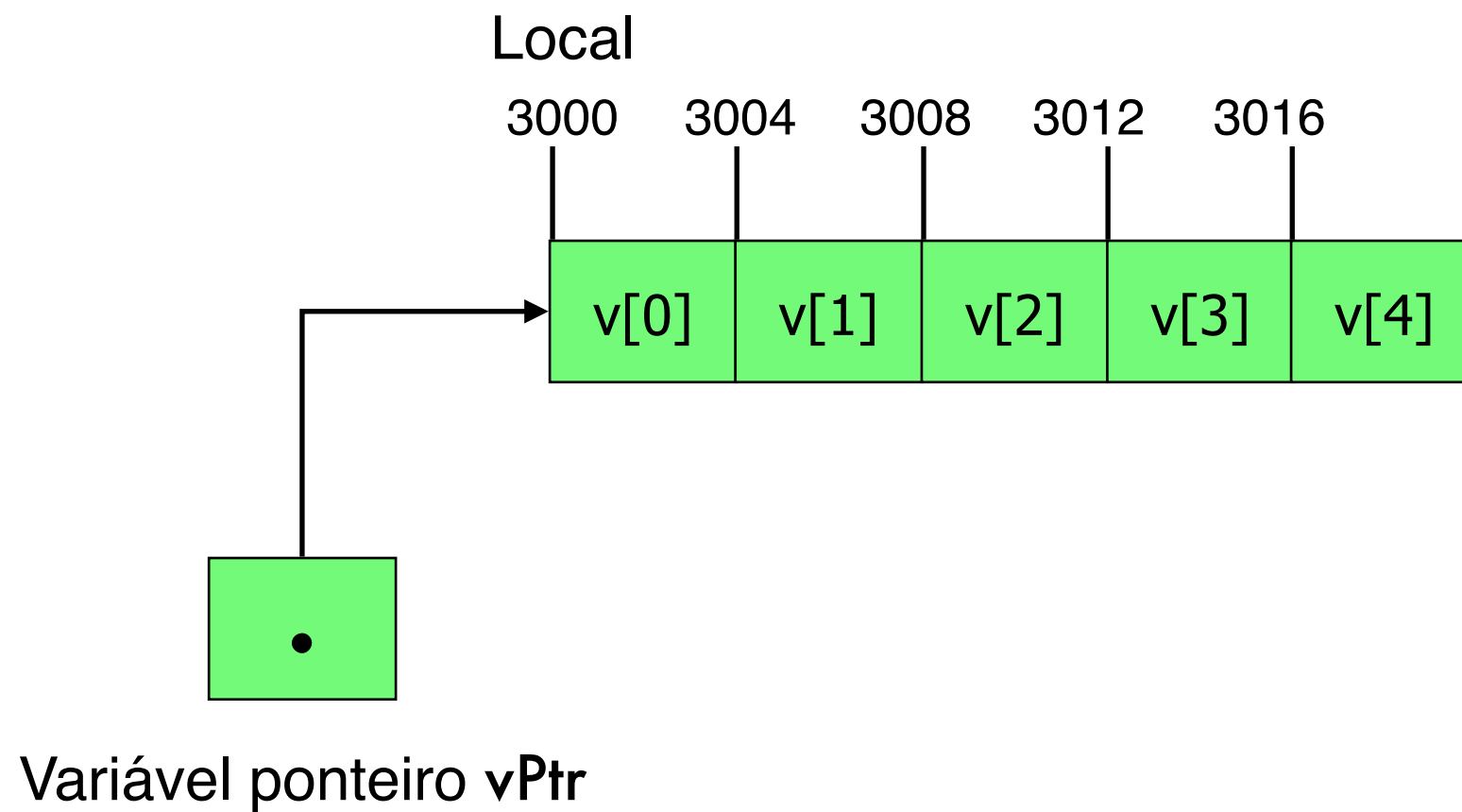

Aritmética em Ponteiros

Aritmética de Ponteiros

- Considere v com 5 elementos em uma máquina onde um **int** possui 4 bytes
- Considere **vPtr** um ponteiro do tipo **int**

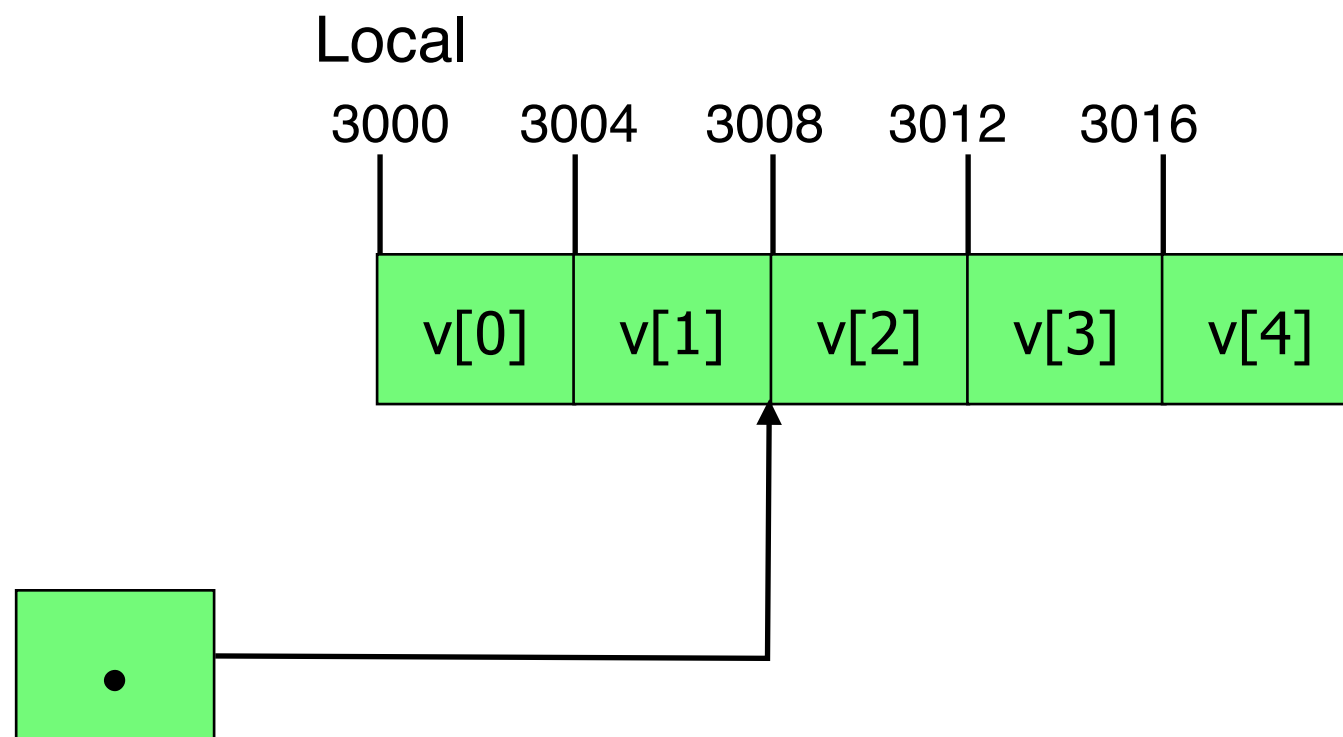
Aritmética de Ponteiros (2)

```
vPtr = v // vPtr aponta para o primeiro elemento v[0]
```



Aritmética de Ponteiros (3)

```
vPtr = v // vPtr aponta para o primeiro elemento v[0]  
vPtr += 2 // soma ao vPtr 2 posições (nesse caso, 8 bytes)
```



Variável ponteiro `vPtr`

Aritmética Convencional $\neq$ Ponteiros

- Na aritmética convencional, $3000 + 2$ resulta o valor 3002
- Frequentemente não é o caso para **aritmética de ponteiros**
 - Quando um inteiro é adicionado ou subtraído de um ponteiro, o ponteiro não é simplesmente incrementado ou decrementado por este inteiro
 - Mas sim pelo inteiro vezes o tamanho do objeto que o ponteiro refere
 - $vPtr + = 1$, então produz 3008 ($3000 + 2 * 4$), assumindo um inteiro armazenado em 4 bytes da memória

Relação entre Ponteiros e Arrays

Relação entre Ponteiros e Arrays

- **Ponteiros** e **arrays** estão intimamente relacionados em C
 - Frequentemente um pode substituir o outro, sem diferença significativa
- **Um nome de um array** pode ser pensado como um **ponteiro constante** (`const *`)
- **Ponteiros** podem utilizar a notação indexação de **arrays** (*array subscripting*)
 - Deslocamento de ponteiro (*pointer offset*)
 - `*(p + 3)`
 - Indexação de array (*array subscripting*)
 - `v[3]`

Exemplo 4: Ponteiro e Array

```
1. #include <stdio.h>
2.
3. int main () {
4.     int v[5] = {123, 345, 678, 146, 121};
5.     int *p;
6.
7.     // p aponta para v (alternativa 1)
8.     p = v;
9.
10.    // p aponta para v (alternativa 2)
11.    p = &v[0];
12.
13.    // modos de acessar o elemento v[3]
14.
15.    // notacao indexacao de array
16.    printf("%d %d\n", v[3], p[3]);
17.
18.    // notacao ponteiro offset
19.    printf("%d %d\n", *(v+3), *(p+3));
20.
21.    return 0;
22. }
```

%
146 146
146 146

Exemplo 5: Copiando String

```
1  /* Fig. 7.21: fig07_21.c
2     Copying a string using array notation and pointer notation. */
3  #include <stdio.h>
4
5  void copy1( char * const s1, const char * const s2 ); /* prototype */
6  void copy2( char *s1, const char *s2 ); /* prototype */
7
8  int main( void )
9  {
10     char string1[ 10 ];          /* create array string1 */
11     char *string2 = "Hello";     /* create a pointer to a string */
12     char string3[ 10 ];          /* create array string3 */
13     char string4[] = "Good Bye"; /* create a pointer to a string */
14
15     copy1( string1, string2 );
16     printf( "string1 = %s\n", string1 );
17
18     copy2( string3, string4 );
19     printf( "string3 = %s\n", string3 );
20
21     return 0; /* indicates successful termination */
22
23 } /* end main */
24
```

Outline

fig07_21.c

(1 of 2)

Exemplo 5: Copiando String (Cont.)

Outline

fig07_21.c

(2 of 2)

```
25 /* copy s2 to s1 using array notation */
26 void copy1( char * const s1, const char * const s2 )
27 {
28     int i; /* counter */
29
30     /* loop through strings */
31     for ( i = 0; ( s1[ i ] = s2[ i ] ) != '\0'; i++ ) {
32         ; /* do nothing in body */
33     } /* end for */
34
35 } /* end function copy1 */
36
37 /* copy s2 to s1 using pointer notation */
38 void copy2( char *s1, const char *s2 )
39 {
40     /* loop through strings */
41     for ( ; ( *s1 = *s2 ) != '\0'; s1++, s2++ ) {
42         ; /* do nothing in body */
43     } /* end for */
44
45 } /* end function copy2 */
```

```
string1 = Hello
string3 = Good Bye
```

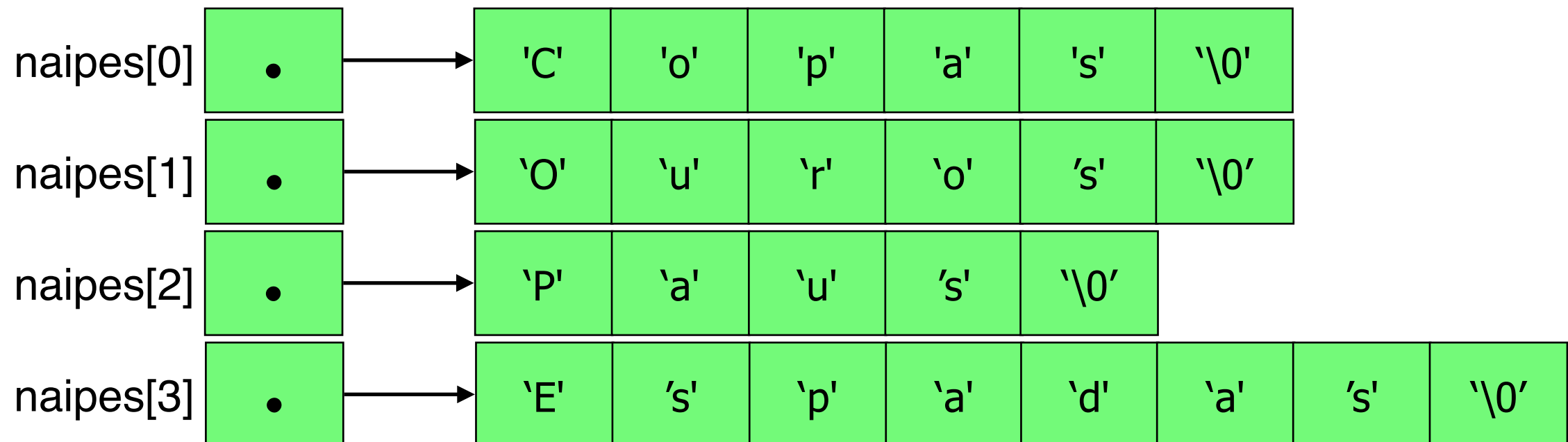
Vetor de Ponteiros

- Vetores podem conter ponteiros
- Um uso comum de **vetor de ponteiros** é para construir um **vetor de strings**
- Cada entrada do vetor é uma string, mas em C uma string é essencialmente um ponteiro para o primeiro caractere

Vetor de Ponteiros

```
char *naipes[4] = {"Copas", "Ouros", "Paus", "Espadas"};
```

- Cada elemento de **suit** é um **char *** (ponteiro para um char)
- As **strings** não são realmente armazenadas no array **suit**, apenas ponteiros para as strings são armazenadas



Representação gráfica do vetor naipes.

Ponteiro para Função

Ponteiro para Função

- **Ponteiro para função** armazena endereço de uma função
 - **Nome da função** é endereço inicial do código que a define
 - Similar a nome de array endereçar o primeiro elemento
- Ponteiro de funções podem ser:
 1. Passado para funções
 2. Armazenado em arrays
 3. Atribuído para outros ponteiros de função

Ordenação por Bolha (*Bubble Sort*)

- Função **bubble sort** recebe um ponteiro para função
 - **Bubble** chama uma função auxiliar que determina se a ordenação é ascendente ou descendente
- Um dos argumento em **bubble sort** é um ponteiro para função:

```
int (*compare) (int a, int b)
```

Exemplo 6: Ordenação por Bolha (1/6)

```
1  /* Fig. 7.26: fig07_26.c
2     Multipurpose sorting program using function pointers */
3  #include <stdio.h>
4  #define SIZE 10
5
6  /* prototypes */
7  void bubble( int work[], const int size, int (*compare)( int a, int b ) );
8  int ascending( int a, int b );
9  int descending( int a, int b );
10
11 int main( void )
12 {
13     int order;    /* 1 for ascending order or 2 for descending order */
14     int counter; /* counter */
15
16     /* initialize array a */
17     int a[ SIZE ] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };
```

Função **bubble** recebe um ponteiro para função como argumento.



Outline

fig07_26.c

(1 of 4)

Exemplo 6: Ordenação por Bolha (2/6)

```
18
19  printf( "Enter 1 to sort in ascending order,\n"
20          "Enter 2 to sort in descending order: " );
21  scanf( "%d", &order );
22
23  printf( "\nData items in original order\n" );
24
25  /* output original array */
26  for ( counter = 0; counter < SIZE; counter++ ) {
27      printf( "%5d", a[ counter ] );
28  } /* end for */
29
```


Exemplo 6: Ordenação por Bolha (3/6)

```
30  /* sort array in ascending order; pass function ascending as an
31     argument to specify ascending sorting order */
32  if ( order == 1 ) {
33      bubble( a, SIZE, ascending );
34      printf( "\nData items in ascending order\n" );
35  } /* end if */
36  else { /* pass function descending */
37      bubble( a, SIZE, descending );
38      printf( "\nData items in descending order\n" );
39  } /* end else */
40
41  /* output sorted array */
42  for ( counter = 0; counter < SIZE; counter++ ) {
43      printf( "%5d", a[ counter ] );
44  } /* end for */
45
46  printf( "\n" );
47
48  return 0; /* indicates successful termination */
49
50 } /* end main */
51
```

Dependendo da escolha do usuário, a função **bubble** usa a função **ascending** ou **descending** para ordenar o array.

Exemplo 6: Ordenação por Bolha (4/6)

```
52 /* multipurpose bubble sort; parameter compare is a pointer to
53    the comparison function that determines sorting order */
54 void bubble( int work[], const int size, int (*compare)( int a, int b ) )
55 {
56     int pass; /* pass counter */
57     int count; /* comparison counter */
58
59     void swap( int *element1Ptr, int *element2ptr ); /* prototype */
60
61     /* loop to control passes */
62     for ( pass = 1; pass < size; pass++ ) {
63
64         /* loop to control number of comparisons per pass */
65         for ( count = 0; count < size - 1; count++ ) {
66
67             /* if adjacent elements are out of order, swap them */
68             if ( (*compare)( work[ count ], work[ count + 1 ] ) ) {
69                 swap( &work[ count ], &work[ count + 1 ] );
70             } /* end if */
71
72         } /* end for */
73
74     } /* end for */
75
76 } /* end function bubble */
77
```

Outline

fig07_26.c

(3 of 4)

Exemplo 6: Ordenação por Bolha (5/6)

Outline

fig07_26.c

(4 of 4)

```
78 /* swap values at memory locations to which element1Ptr and
79    element2Ptr point */
80 void swap( int *element1Ptr, int *element2Ptr )
81 {
82     int hold; /* temporary holding variable */
83
84     hold = *element1Ptr;
85     *element1Ptr = *element2Ptr;
86     *element2Ptr = hold;
87 } /* end function swap */
88
89 /* determine whether elements are out of order for an ascending
90    order sort */
91 int ascending( int a, int b )
92 {
93     return b < a; /* swap if b is less than a */
94
95 } /* end function ascending */
96
97 /* determine whether elements are out of order for a descending
98    order sort */
99 int descending( int a, int b )
100 {
101     return b > a; /* swap if b is greater than a */
102
103 } /* end function descending */
```

Passar para **bubble** a função **ascending**
irá apontar o programa aqui

Passar para **bubble** a função **descending**
irá apontar o programa aqui

Exemplo 6: Ordenação por Bolha (6/6)

Outline

```
Enter 1 to sort in ascending order,  
Enter 2 to sort in descending order: 1
```

```
Data items in original order
```

```
2    6    4    8    10   12   89   68   45   37
```

```
Data items in ascending order
```

```
2    4    6    8    10   12   37   45   68   89
```

```
Enter 1 to sort in ascending order,  
Enter 2 to sort in descending order: 2
```

```
Data items in original order
```

```
2    6    4    8    10   12   89   68   45   37
```

```
Data items in descending order
```

```
89   68   45   37   12   10    8    6    4    2
```

Acesso Campos Ponteiro para Struct

```
#include <stdio.h>

struct ponto2d {
    double x, y;
};

int main() {
    struct ponto2d p1 = {3.0, 4.0}, *p2;

    printf("p1.x = %.2lf, p1.y = %.2lf\n", p1.x, p1.y);
    p2 = &p1;

    // alternativa 1: *p2
    printf("*p2.x = %.2lf, *p2.y = %.2lf\n", (*p2).x, (*p2).y);

    // alternativa 2: ->p2
    printf("p2->x = %.2lf, p2->y = %.2lf\n", p2->x, p2->y);

    return 0;
}
```

Referências

- DEITEL, C Como Programar 6ª Edição. Pearson;
- FEOFILOFF, P. Algoritmos em Linguagem C, 1. ed. Rio de Janeiro: Elsevier, 2008;
- PIVA, D.J. et al. Algoritmos e programação de computadores. Rio de Janeiro: Elsevier, 2012;
- WIRTH, Niklaus. Algoritmos e estrutura de dados. Rio de Janeiro, RJ: LTC, 2009. 255p;